

Tasky API V2 Documentation

Version: 2.0.0

Base URL: <https://tasky.pl-coding.com>

Authentication: JWT Bearer Token

Overview

This will serve as documentation for all the API endpoints needed to build Tasky.

Please note that this API has been written with care, however, if you still find a bug, reporting it is greatly appreciated.

 **Base URL:** <https://tasky.pl-coding.com>

 **Figma Mockups:** [Tasky Design](#)

Authentication

All endpoints require an API key to be attached. It needs to be attached as `x-api-key` header, like this

Header name	Header value
x-api-key	<your API key>

All protected endpoints require a valid JWT token in the Authorization header:

```
Authorization: Bearer <your-jwt-token>
```

Authentication Flow

1. **Register** → Create an account
2. **Login** → Receives `accessToken`, `refreshToken`, `userId`, and `username`
3. **Use access token** for authenticated requests (lifetime: 60mins)
4. **Refresh token** via `/auth/refresh` to get new token using your stored `refreshToken`
5. **Logout** → Invalidates the refresh token



Testing the OAuth mechanism

Use the test account below to test short-lived tokens (valid for 10 seconds):

- **Email:** `debug@debug.com`
- **Password:** `Debug12345`



Rate Limiting

To avoid accidental bursts:

- 50 requests/min
- 1000 requests/hour

Returns `429 Too Many Requests` when limits exceeded



Last-Write-Wins Conflict Resolution

CRUD operations use timestamp-based conflict resolution via the `updatedAt` field:

- When updating an event, the client sends its `updatedAt` timestamp
- Server compares with its stored `updatedAt` value
- If server timestamp is newer, client changes are rejected and server returns current state
- If client timestamp is newer or equal, changes are accepted
- This prevents data loss while maintaining eventual consistency across devices



API Endpoints



Authentication Endpoints

Register User

Register a new user

POST /auth/register
Content-Type: application/json

```
{  
  "fullName": "John Doe",    (String)  
  "email": "john@example.com", (String)  
  "password": "MySecurePass123" (String)  
}
```

Validation:

- `fullName` : 4–50 characters
- `email` : Valid format
- `password` : ≥ 9 chars, includes uppercase, lowercase, and number

Response: 200 OK (No content)

Errors:

- 409 CONFLICT: User already exists
- 400 BAD REQUEST: Validation errors

Login

Login with email/password, optional debug header. When the `Debug` header is set to true, the generated token will only be valid for 30 seconds.

POST /auth/login
Content-Type: application/json
Debug: true

```
{  
  "email": "john@example.com", (String)  
  "password": "MySecurePass123" (String)  
}
```

Response:

```
{
  "accessToken": "eyJh...QXVCJ9", (String)
  "refreshToken": "dGhpcy...G9rZW4=", (String)
  "username": "John Doe", (String)
  "userId": "123e4567-e89b-12d3-a456-426614174000", (String)
  "accessTokenExpirationTimestamp": 1759164834678 (Long)
}
```

Refresh Token

Refresh JWT tokens using refresh token, optional debug header. When the **Debug** header is set to true, the generated token will only be valid for 30 seconds.

```
POST /auth/refresh
Content-Type: application/json
Debug: true

{
  "refreshToken": "dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4=" (String)
}
```

Response:

```
{
  "accessToken": "...", (String)
  "refreshToken": "..." (String)
}
```

Errors:

- 401 UNAUTHORIZED: Invalid refresh token

Logout

Invalidate refresh token

```
POST /auth/logout
Content-Type: application/json
Authorization: Bearer <token>
```

```
{
  "refreshToken": "..." (String)
}
```

Response: 200 OK (No content)

👉 ***Logout functionality should be disabled in offline mode to avoid synchronization complexities.***

Agenda Management

Get Daily Agenda

```
GET /agenda?time=<timestamp>
Authorization: Bearer <token>
```

Response:

Returns events, tasks, and reminders for a specific date.

```
{
  "events": [...events],    (List<Event>)
  "tasks": [...tasks],     (List<Task>)
  "reminders": [...reminders] (List<Reminder>)
}
```

Notes:

- time must be ISO 8601 format
- Future times are not allowed

Get Full Agenda

Get all user's agenda items regardless of date. Used for syncing local cache.

```
GET /fullAgenda
Authorization: Bearer <token>
```

Response

```
{
  "events": [...events],    (List<Event>)
  "tasks": [...tasks],     (List<Task>)
  "reminders": [...reminders] (List<Reminder>)
}
```

Sync Agenda (Batch Delete)

Synchronizes locally deleted agenda items with the API. For creating/updating items, use individual POST/PUT endpoints.

```
POST /syncAgenda
Content-Type: application/json
Authorization: Bearer <token>

{
  "deletedEventIds": [...event_IDs],    (List<String>)
  "deletedTaskIds": [...task_IDs],     (List<String>)
  "deletedReminderIds": [...reminder_IDs], (List<String>)
}
```

Response: 200 OK (No content)

Synchronization Strategy

Two-Tier Sync Approach

The API implements a two-tier synchronization strategy:

1. Bulk Deletions: Use `POST /syncAgenda` for efficiently syncing deleted items across all agenda types
2. Individual Operations: Use standard CRUD endpoints (`POST` , `PUT` , `DELETE`) for creating and updating items

Why syncAgenda is Deletion-Only

The syncAgenda endpoint handles only deletions because:

- Complexity Management: Full sync with creations/updates would require handling massive payloads when events contain multiple photos and attendees
- Network Efficiency: Large sync requests could timeout or fail, especially with photo uploads
- Error Isolation: Individual operations allow for granular error handling and retry logic
- Memory Constraints: Mobile devices can't handle large sync payloads efficiently

Task Management

Create Task

Create task with title, description, time, completion status

`POST /task`

Content-Type: application/json

Authorization: Bearer <token>

```
{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Complete API documentation", (String)
  "description": "Write comprehensive API docs", (String)
```

```
"time": "2024-01-15T14:00:00Z",      (String)
"remindAt": "2024-01-15T13:45:00Z",  (String)
"updatedAt": "2024-01-15T12:00:00Z", (String)
"isDone": false                      (Boolean)
}
```

Validation:

- `id`: Valid UUID
- `title`: Required, cannot be blank
- Dates: ISO 8601 format

Response:

```
{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Complete API documentation",      (String)
  "description": "Write comprehensive API docs", (String)
  "time": "2024-01-15T14:00:00Z",            (String)
  "remindAt": "2024-01-15T13:45:00Z",        (String)
  "isDone": true                             (Boolean)
}
```

Errors:

- 409 CONFLICT: Task with ID already exists

Update Task

Update existing task

```
PUT /task
Content-Type: application/json
Authorization: Bearer <token>

{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Updated task title",                  (String)
}
```



```
...  
}
```

Response:

```
{  
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)  
  "title": "Updated task title", (String)  
  "description": "Write comprehensive API docs", (String)  
  "time": "2024-01-15T14:00:00Z", (String)  
  "remindAt": "2024-01-15T13:45:00Z", (String)  
  "isDone": true (Boolean)  
}
```

Errors:

- 404 NOT FOUND: Task doesn't exist
- 403 FORBIDDEN: Task doesn't belong to user

Get Task

Get task by client UUID (user must own task)

```
GET /task/{taskId}  
Authorization: Bearer <token>
```

Path parameters:

- `taskId`: UUID of the task

Response:

```
{  
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)  
  "title": "Updated task title", (String)  
  "description": "Write comprehensive API docs", (String)  
  "time": "2024-01-15T14:00:00Z", (String)  
  "remindAt": "2024-01-15T13:45:00Z", (String)  
}
```

```
"isDone": true                (Boolean)
}
```

Errors:

- 404 NOT FOUND: Task doesn't exist
- 403 FORBIDDEN: Task doesn't belong to user

Delete Task

Delete user's task

```
DELETE /task/{taskId}?deletedAt=2024-01-15T16:00:00Z
Authorization: Bearer <token>
```

Path parameters:

- `taskId`: UUID of the task

Query parameters:

- `deletedAt`: Optional ISO 8601 timestamp (defaults to current time)

Response: 200 OK (No content)

Errors:

- 400 BAD REQUEST: Invalid UUID format or date format
- 404 NOT FOUND: Task doesn't exist
- 403 FORBIDDEN: Task doesn't belong to user



17 Event Management

Create Event

Create event with attendees and photo upload support

```
POST /event
Content-Type: application/json
Authorization: Bearer <token>

{
```

```

"id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
"title": "Team Meeting", (String)
"description": "Weekly sync meeting", (String)
"from": "2024-01-15T10:00:00Z", (String)
"to": "2024-01-15T11:00:00Z", (String)
"remindAt": "2024-01-15T09:30:00Z", (String)
"attendeelds": ["507f1f77bcf86cd799439012"], (List<String>)
"photoKeys": ["photo01", "photo02"], (List<String>)
"updatedAt": "2024-01-15T09:00:00Z" (String)
}

```

Validation:

- `id` : Valid UUID (client-generated)
- `title` : Required, cannot be blank
- `from` : Must be before `to`
- `to` : Must be after `from`
- `remindAt` : Must be before `from`
- `attendeelds` : List of valid user IDs
- `photoKeys` : List of photo keys for S3 upload
- `updatedAt` : ISO 8601 format, cannot be in future

Response:

```

{
  "event": {
    "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
    "title": "Team Meeting", (String)
    "description": "Weekly sync meeting", (String)
    "from": "2024-01-15T10:00:00Z", (String)
    "to": "2024-01-15T11:00:00Z", (String)
    "remindAt": "2024-01-15T09:30:00Z", (String)
    "hostId": "<Event creator ID>", (String)
    "isUserEventCreator": true, (Boolean)
    "attendees": [...attendees], (List<Attendee>)
    "photoKeys": [...downloadable photos], (List<Photo>)
  },
}

```

```

"uploadUrls": [
  {
    "photoKey": "photo0", (String)
    "uploadKey": "4d0daede-fbe9-449b-ae26-c4337e3515a4", (String)
    "url": "<URL that can be used to upload image>" (String)
  }
]
}

```

Attendee:

```

{
  "email": "<Attendee email>", (String)
  "username": "<Attendee full name>", (String)
  "userId": "<Attendee user ID>", (String)
  "eventId": "<Event the attendee belongs to>", (String)
  "isGoing": true/false, (Boolean)
  "remindAt": "2024-01-15T09:30:00Z" (String)
}

```

Photo:

```

{
  "key": "<Unique key of the photo>", (String)
  "url": "https://test.com/photo.png" (String)
}

```

Errors:

- 400 BAD REQUEST: Validation errors (invalid timestamps, blank title, invalid UUID)



Event Creation & Photo Upload Flow

Two-Phase Event Creation

1. Create Event: `POST /event`
 - Client sends event data including `photoKeys` array
 - Server returns event object + upload URLs
 - Photos are not yet accessible/visible
2. Confirm Upload: `POST /event/{eventId}/confirm-upload`
 - Client uploads photos using the upload URLs
 - Client confirms successful uploads with uploaded keys
 - Server validates uploads and generates download URLs
 - Photos become accessible in the event

Update Event

- Update event with role-based permissions:
 - Host: Can modify everything (`title` , `time` , `attendees` , `photos`)
 - Attendee: Can only update `isGoing` status and `remindAt`
- Conflict resolution using `updatedAt` timestamp

`PUT /event/{eventId}`

Content-Type: application/json

Authorization: Bearer <token>

```
{
  "title": "Updated Meeting Title",          (String)
  "description": "Updated description",      (String)
  "from": "2024-01-15T10:00:00Z",          (String)
  "to": "2024-01-15T11:00:00Z",            (String)
  "remindAt": "2024-01-15T09:30:00Z",      (String)
  "attendeeIds": ["507f1f77bcf86cd799439012"], (List<String>)
  "newPhotoKeys": ["photo[0-9]"],          (List<String>)
  "deletedPhotoKeys": ["photo_unique_id"],  (List<String>)
  "isGoing": true,                          (Boolean)
```

```
"updatedAt": "2024-01-15T09:30:00Z"      (String)
}
```

Path Parameters:

- `eventId`: UUID of the event

Response:

```
{
  "event": {
    "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
    "title": "Team Meeting", (String)
    "description": "Weekly sync meeting", (String)
    "from": "2024-01-15T10:00:00Z", (String)
    "to": "2024-01-15T11:00:00Z", (String)
    "remindAt": "2024-01-15T09:30:00Z", (String)
    "hostId": "<Event creator ID>", (String)
    "isUserEventCreator": true, (Boolean)
    "attendees": [...attendees], (List<Attendee>)
    "photoKeys": [...downloadable photos], (List<Photo>)
  },
  "uploadUrls": [
    {
      "photoKey": "photo0", (String)
      "uploadKey": "4d0daede-fbe9-449b-ae26-c4337e3515a4", (String)
      "url": "<URL that can be used to upload image>" (String)
    }
  ]
}
```

Attendee:

```
{
  "email": "<Attendee email>", (String)
  "username": "<Attendee full name>", (String)
  "userId": "<Attendee user ID>", (String)
  "eventId": "<Event the attendee belongs to>", (String)
  "isGoing": true/false, (Boolean)
}
```

```
"remindAt": "2024-01-15T09:30:00Z"      (String)
}
```

Photo:

```
{
  "key": "<Unique key of the photo>", (String)
  "url": "https://test.com/photo.png" (String)
}
```

Errors:

- 404 NOT FOUND: Event doesn't exist

 **Note that the same two-phase flow should be followed when updating an event.**

Get Event

Get event details by UUID

```
GET /event/{eventId}
Authorization: Bearer <token>
```

Path Parameters:

- `eventId`: UUID of the event

Response:

```
{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Team Meeting", (String)
  "description": "Weekly sync meeting", (String)
  "from": "2024-01-15T10:00:00Z", (String)
  "to": "2024-01-15T11:00:00Z", (String)
  "hostId": "507f1f77bcf86cd799439011", (String)
  "isUserEventCreator": true, (Boolean)
  "attendees": [...], (List<Attendee>)
```

```
"photoKeys": [...] (List<Photo>)  
}
```

Errors:

- 404 NOT FOUND: Event doesn't exist

Delete Event

Delete event (host only)

```
DELETE /event?eventId=<event_UUID>&deleteAt=<timestamp>  
Authorization: Bearer <token>
```

Path parameters:

- `eventId`: UUID of the event

Query parameters:

- `deleteAt`: Optional ISO 8601 timestamp (defaults to current time)

Response: 200 OK (No content)

Errors:

- 400 BAD REQUEST: Invalid UUID format or date format
- 404 NOT FOUND: Event doesn't exist

Confirm Upload

Confirms that photos have been uploaded using the upload URL recieved. Once confirmed, the photos will have a download URL generated and attached to the event (host only).

```
POST /event/{eventId}/confirm-upload  
Content-Type: application/json  
Authorization: Bearer <token>  
  
{  
  "uploadedKeys": ["<photo_unique_id>", "<photo_unique_id>"] (List<String>)  
}
```


Path Parameters:

- `eventId`: UUID of the event

Response:

```
{
  "event": {
    "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
    "title": "Team Meeting", (String)
    "description": "Weekly sync meeting", (String)
    "from": "2024-01-15T10:00:00Z", (String)
    "to": "2024-01-15T11:00:00Z", (String)
    "remindAt": "2024-01-15T09:30:00Z", (String)
    "hostId": "<Event creator ID>", (String)
    "isUserEventCreator": true, (Boolean)
    "attendees": [...attendees], (List<Attendee>)
    "photoKeys": [...downloadable photos], (List<Photo>)
  },
  "uploadUrls": [
    {
      "key": "4d0daede-fbe9-449b-ae26-c4337e3515a4", (String)
      "url": "<URL that can be used to upload image>" (String)
    }
  ]
}
```

Attendee:

```
{
  "email": "<Attendee email>", (String)
  "username": "<Attendee full name>", (String)
  "userId": "<Attendee user ID>", (String)
  "eventId": "<Event the attendee belongs to>", (String)
  "isGoing": true/false, (Boolean)
  "remindAt": "2024-01-15T09:30:00Z" (String)
}
```

Photo:

```
{
  "key": "<Unique key of the photo>", (String)
  "url": "https://test.com/photo.png" (String)
}
```

Errors:

- 400 BAD REQUEST: Invalid upload keys
- 404 NOT FOUND: Event doesn't exist

Reminder Management

Create Reminder

Create reminder with title, description, time

```
POST /reminder
Content-Type: application/json
Authorization: Bearer <token>

{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Doctor Appointment", (String)
  "description": "Annual checkup", (String)
  "time": "2024-01-15T16:00:00Z", (String)
  "remindAt": "2024-01-15T15:30:00Z", (String)
  "updatedAt": "2024-01-15T15:00:00Z" (String)
}
```

Validation:

- **id**: Valid UUID
- **title**: Required, cannot be blank
- Dates: ISO 8601 string format (stored as Instant in responses)
- **updatedAt**: Cannot be in future

Response:

```
{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Doctor Appointment", (String)
  "description": "Annual checkup", (String)
  "time": "2024-01-15T16:00:00Z", (String)
  "remindAt": "2024-01-15T15:30:00Z" (String)
}
```

Errors:

- 400 BAD REQUEST: Validation errors (invalid UUID, blank title, invalid date format)

Update Reminder

Update existing reminder

```
PUT /reminder
Content-Type: application/json
Authorization: Bearer <token>

{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Updated Reminder", (String)
  "description": "Updated description", (String)
  "time": "2024-01-15T17:00:00Z", (String)
  "remindAt": "2024-01-15T16:30:00Z", (String)
  "updatedAt": "2024-01-15T16:00:00Z" (String)
}
```

Response:

```
{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Doctor Appointment", (String)
  "description": "Annual checkup", (String)
  "time": "2024-01-15T16:00:00Z", (String)
}
```

```
"remindAt": "2024-01-15T15:30:00Z"      (String)
}
```

Errors:

- 400 BAD REQUEST: Validation error
- 404 NOT FOUND: Reminder doesn't exist

Get Reminder

Get reminder by UUID

```
GET /reminder/{reminderId}
Authorization: Bearer <token>
```

Path Parameters:

- `reminderId`: UUID of the reminder

Response:

```
{
  "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479", (String)
  "title": "Doctor Appointment", (String)
  "description": "Annual checkup", (String)
  "time": "2024-01-15T16:00:00Z", (String)
  "remindAt": "2024-01-15T15:30:00Z" (String)
}
```

Errors:

- 400 BAD REQUEST: Invalid UUID format
- 404 NOT FOUND: Reminder doesn't exist

Delete Reminder

Delete user's reminder

```
DELETE /reminder/{reminderId}?deletedAt=2024-01-15T17:00:00Z
Authorization: Bearer <token>
```

Path Parameters:

- `reminderId` : UUID of the reminder

Query Parameters:

- `deletedAt` : Optional ISO 8601 timestamp (defaults to current time)

Response: 200 OK (No content)

Errors:

- 400 BAD REQUEST: Invalid UUID format or date format
- 404 NOT FOUND: Reminder doesn't exist

Attendee Management

Check User Existence

Check if user exists by email for event invitations

```
GET /attendee?email=user@example.com
Authorization: Bearer <token>
```

Query Parameters:

- `email` : Email address to check

Response:

```
{
  "email": "user@example.com",      (String)
  "fullName": "John Doe",          (String)
  "userId": "507f1f77bcf86cd799439012" (String)
}
```

Errors:

- 400 BAD REQUEST: Invalid email format
- 404 NOT FOUND: User not found
- 409 Conflict if user tries to add themselves

Remove Attendee from Event

Remove authenticated user from event's attendee list

DELETE /attendee?eventId=f47ac10b-58cc-4372-a567-0e02b2c3d479
Authorization: Bearer <token>

Query Parameters:

- `eventId`: UUID of the event

Response: 200 OK (No content)

Errors:

- 400 BAD REQUEST: Invalid UUID format
- 404 NOT FOUND: Event doesn't exist or user is not an attendee

Error Handling

Code	Meaning
200	OK - Request successful
400	Bad Request - Invalid request format or validation errors
401	Unauthorized - Authentication required or invalid token
403	Forbidden - Insufficient permissions
404	Not Found - Resource not found
409	Conflict - Resource already exists
500	Internal Server Error - Server error

Error Response Format

The details field is only sent for a validation error.

```
{
  "status": 400,
  "reason": "Validation Error",
  "details": [
    "Validation failed #1",
    "Validation failed #2",
  ]
}
```

Best Practices

1. **Content-Type:** Always include `Content-Type: application/json` for POST/PUT requests
 2. **HTTP Methods:** Use correct verbs (GET, POST, PUT, DELETE)
 3. **Authentication:** Include `Authorization: Bearer <token>` header for protected endpoints
 4. **UUIDs:** Ensure all IDs are valid UUIDs generated client-side
 5. **Error Handling:** Always check status codes and handle errors gracefully
 6. **Timestamps:** Use ISO 8601 format (`2024-01-15T12:00:00Z`)
-